low end average 72

Ed Scaife -2048 Write-Up

Edward Scaife –2048 Write-up

Main Body and general info

The main body of my program opens the file and prints out the initial configuration and then enters a do while loop. The loop can only terminate when the result returned from function checkifover() is true. I choose a do while loop as opposed to a while loop because I was not checking if the player had lost until the very end of the loop and if they had lost, there would be no point returning to the top of the loop to check conditions.

The droptwo() function and the printout() function are both dependant on variable x. Variable x is returned as true if the movement function called by the user causes movement within the grid to occur, otherwise it is returned false. This effectively means that if an input is entered and it causes no movement on the grid, then a new grid will not be printed and a 2 will not be dropped onto the grid.

The structure that I chose to use for my grid was a two-dimensional array. This is advantageous compared to flat array as it effectively assigns each possible position with a coordinate rather than just a single number. I found that this made it much more easy to visualise and therefore debug. Using the 2d array also had the advantage that it made it so that all of my movement functions could have near enough the same format(I.e. the increment I, then increment j format).

Movement functions

My movement control is split into two separate functions direction() and directiontwo(). Direction() moves all the tiles in the direction required, provided they are able to move; and directiontwo() merges two adjacent identical numbers. X is returned from all of these function and is set to true only if movement occurs.

These functions are called in the order: shift, merge, shift again. This, I believe, is the optimal order to run them in. Originally, I had it in order merge, shift, merge, shift. However, this failed to work correctly in situations where 4 items in a row are identical(e.g. 2 2 2 2).

Each different direction function reads the array in a different way(e.g. up() and uptwo() start in the top left and works going down first, whereas right() and righttwo() start in the top right and work by going right first.) This is a useful, if not necessary feature as it allows all the possible shifting moves to be done in a single run through of the function, instead of having to restart the function each time a piece is moved.

End Game Check

The checkifover() function is the last thing to run in each loop. This function is effectively split into two parts. The first part checks if the grid contains any zeroes. If it does, the function returns false. The second part checks if any points are adjacent to any others with the same value. If any result is adjacent to one identical to itself, the function returns false. Only if the program reaches the very end of the function does the function return true and the game is lost. Since the state of having zeroes is more a more common reason to continue(at least in the early game), this part is arranged chronologically first.

I originally set this function up to just simulate all possible moves in the next turn and returning true if all functions failed to produce a result. However, this was extremely inefficient worked unreliably. It also required the grid to be backed up and then restored, which in itself took a great many cycles.

56 Below average

Sabatino

This program allows to play the game 2048.

The main is composed by an if that asks if the player has a saved game, asks for name of said files and loads said file (if a negative answer or a non valid file name is provided, then a standard game is created), and a while cycle (from which you can exit only if you loose) that asks asks for a direction (with subcicle if inserted command is not valid), calls the function move and shows the grid (through a separate function).
My objective was to keep as little as possible in the main, while keeping there the process of opening the input file.
Main is also the only place where the "show" (shows the grid) function is called.

The "move" function takes the input from the user (through main) and then either calls the mv function, giving it a bool input ("col"), which says if we are shifting columns or rows and an integer "dir" input (that either is +1 or -1) that says if we are going in "upward" or "downward" (if we think about a single array line, "upward" is moving the numbers toward lin[0], "downward" towards lin[3]).

The function "mv" has a "for" cycle, to shift all the columns and, deepening on whether "col" is true or false, calls the ecol (extract column) function or the erow (extract row) function.
These functions extract the row/column given given by the for cycle in "mv", and place if in a an array "lin[4]". (if the dir variables previously passed is -1, then the functions also invert lin[4]).

The lin[4]is then passed to the line function, which acts as follows:

1. checks how many numbers different from 0 are in the line and saves it into a variable n
2. enters a while loop from which it can enter only when all the numbers different from 0 are in the first n position of the line
3. checks every cell (except place number 3), if the number in the cell is 0 then switches the cell value with the next cell.
4. checks the first n cells in the line and saves the value to check against n

Then Line proceeds to check the numbers different from 0 (now all in the beginning of the line) and if it finds two consecutive equal numbers it sums them, before moving over (places a 0 in the slot of the consecutive number that will be shifted over with the cycle).
This operations is done in a for cycle, an terminates once the counter reaches n.

Then the ecol/erow functions reinserts lin[4] in the original array. If there is at least a difference between the two, then the "moved" variable will go to 1 in ecol/erow, and then in mv.
If by the time the "mv" for cycle is over mv is not to 1, then the direction given was not valid, so mv will return false to "move", which in turn will return false to "main", asking the player for a new direction without showing the grid.

The last function to mention is the "chkwin" function. It controls the while loop in the main, and becomes false once the player has no available moves. To check if the players as any available moves it uses a double for cycle, that returns true if it finds a 0 or two consecutive equal numbers (using function chkeq).

The aim of my design was to keep the shifting process in a single function and to make as little redundancies as possible (that was the aim of the whole extraction process, that allowed the "line" function to ignore the effective direction of the movement and whether the line was a row or a column). Unfortunately I had no time to debug properly, so many errors are still present in the program.